defensiva JWT JSON Web Token autenticación

Qué es un JWT (JSON Web Token): cómo funciona, vulnerabilidades y buenas prácticas

Qué es un JWT, estructura header/payload/signature, algoritmos HS256 y RS256, bugs habituales (alg=none, kid injection, weak secret) y buenas prácticas de seguridad.

Secra 8 de mayo de 2026 11 min de lectura

Un **JWT (JSON Web Token)** es **un formato compacto y firmado de token portador (bearer) que transporta información autenticada entre dos sistemas, normalmente entre un servidor y un cliente web o móvil**. Su uso más extendido es la

CONTENIDO

Qué es un JWT y para qué se usa

Estructura de un JWT: header, payload, signature

Header

Payload

Signature

autenticación: el servidor genera un JWT al hacer login del usuario, el cliente lo guarda y lo envía en cada petición posterior, y el servidor verifica la firma para confirmar que el token sigue siendo válido sin necesidad de mantener una sesión activa en base de datos. Está estandarizado en el RFC 7519 y es la base de OAuth 2.0, OpenID Connect y la mayoría de arquitecturas API modernas.

Esta guía explica qué es un JWT, su estructura interna, los algoritmos de firma habituales, los bugs que más se explotan en pentesting (alg=none, kid injection, weak secret, JWT confusion), las diferencias con la sesión tradicional y las buenas prácticas defensivas vigentes.

Qué es un JWT y para qué se usa

Un JWT es **una cadena de texto**

Algoritmos de firma habituales

Bugs habituales en JWT que aparecen en pentesting

1. `alg: none`

2. Algoritmo asimétrico tratado como simétrico

3. Weak secret en HS256

4. Inyección en `kid`

5. JWT confusion entre dominios

6. Tokens sin expiración o expiración eterna

7. Firma HS256 con la palabra "secret" como clave

JWT vs sesión tradicional con cookie

Buenas prácticas de seguridad con JWT

JWT y compliance

Preguntas frecuentes

¿JWT es seguro por sí mismo?

¿Cuál es la diferencia entre JWT y OAuth 2.0?

¿Conviene cifrar el payload (JWE) en lugar de sólo firmarlo (JWS)?

¿Cuánto debe durar un JWT?

¿Cómo se revoca un JWT antes de su expiración?

¿Dónde guardo el JWT en el cliente: cookie o localStorage?

Recursos relacionados

ASCII que codifica información estructurada (claims) y la firma con un secreto o una clave privada. El receptor verifica la firma y, si es válida, confía en los claims.

Casos de uso típicos:

- **Autenticación**: tras el login, el servidor emite un JWT que el cliente envía en el header `Authorization: Bearer <token>` de cada petición posterior.
- **Autorización**: el JWT contiene los roles o permisos del usuario; el servidor decide qué puede hacer sin volver a consultar la base de datos.
- **Intercambio entre servicios**: en arquitecturas microservicios, un servicio firma un JWT para autenticarse contra otro.
- **Single Sign-On (SSO)**: el proveedor de identidad emite un JWT que vale en todos los servicios federados (OpenID Connect). En entornos

JWT en Secra

> corporativos legacy, la alternativa basada en XML es SAML. Los kits de phishing AitM modernos atacan precisamente el flujo SSO para capturar tokens, técnica documentada en la guía de Man in the Middle.

Lo que hace útil al JWT no es la criptografía (es estándar), sino el formato compacto y autocontenido: cabe en una cookie o un header HTTP, se transmite eficientemente y permite verificación sin estado en el servidor.

Estructura de un JWT: header, payload, signature

Un JWT son tres bloques separados por puntos:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ
```

Cada bloque es JSON codificado en Base64URL.

Header

Define el algoritmo de firma y el tipo de token. Ejemplo decodificado:

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "key-2026-01"
}
```

Campos típicos: `alg` (algoritmo, obligatorio), `typ` (tipo, opcional), `kid` (key ID, identifica la clave usada cuando hay rotación).

Payload

Contiene los claims (afirmaciones sobre el usuario o la sesión). Hay claims registrados (`sub`, `iss`, `aud`, `exp`, `iat`, `nbf`, `jti`) y claims custom de la aplicación.

```
{
```

```
  "sub": "1234",
  "role": "admin",
  "exp": 1747900800,
  "iat": 1747897200
}
```

Importante: **el payload no está cifrado, sólo codificado**. Cualquiera con el token puede leer el contenido. No metas secretos, contraseñas ni datos sensibles que no deban ser visibles para el cliente.

Signature

Es la firma criptográfica del header y el payload concatenados. Se calcula con:

```
HMAC-SHA256(base64url(header) + "."
```

para algoritmos simétricos (HS256), o con clave privada RSA/ECDSA para algoritmos asimétricos (RS256, ES256).

La firma es lo que garantiza que ni

header ni payload se han modificado tras la emisión. Sin firma válida, el servidor rechaza el token.

Algoritmos de firma habituales

Los algoritmos más usados en producción y sus diferencias operativas:

Algoritmo	Tipo
HS256	Simétrico (HMAC
HS384 / HS512	Simétrico, hash m
RS256	Asimétrico (RSA +
RS384 / RS512	Asimétrico, hash r
ES256	Asimétrico (ECDS
ES384 / ES512	ECDSA con curva
EdDSA	Curva Ed25519
none	**Sin firma**

Decisión rápida: HS256 si el mismo servicio firma y verifica; RS256 o ES256 si hay separación entre quien emite y quien verifica.

Bugs habituales en JWT que aparecen en pentesting

Son la categoría que con más frecuencia aparece en auditorías de pentesting de APIs y aplicaciones web:

1. `alg: none`

El RFC permite el algoritmo `none` para tokens sin firma. Bibliotecas mal configuradas aceptan tokens con `alg: none` en el header y validan sin verificar firma. El atacante elimina la firma y modifica el payload libremente:

```
{"alg": "none", "typ": "JWT"}.{"sub
```

Defensa: rechazar explícitamente `alg: none` en el código de validación. Configurar la lista blanca de algoritmos aceptados.

2. Algoritmo asimétrico tratado como simétrico

Si el servidor está configurado para validar RS256 pero la biblioteca usa la clave pública como secret de HS256, un atacante puede firmar tokens con la clave pública (que es accesible) y el servidor los acepta como HMAC válido. Vulnerabilidad histórica en muchas librerías; sigue apareciendo en código legacy.

Defensa: validar que el algoritmo del header coincide con el algoritmo esperado para esa clave.

3. Weak secret en HS256

Si la clave HMAC es corta o predecible (la frase del fundador, "secret", `password123`), un atacante extrae la firma de un token capturado y la rompe offline con

hashcat o john the ripper en horas o minutos. Una vez crackeado el secret, puede firmar cualquier token.

Defensa: secret de al menos 32 bytes aleatorios (recomendado 64). Generar con `openssl rand -base64 64`.

4. Inyección en `kid`

El campo `kid` (key ID) selecciona la clave concreta que el servidor usa para verificar. Si el código consulta el `kid` en una base de datos sin sanitizar, abre SQL injection. Si lee un fichero local (`/keys/{kid}.pem`), abre path traversal:

```
"kid": "../../../../dev/null"
```

Algunos exploits hacen que el servidor valide la firma contra un fichero conocido del sistema, permitiendo al atacante predecir la firma.

Defensa: validar `kid` contra una lista blanca, nunca usarlo directo en queries o paths.

5. JWT confusion entre dominios

Si el `iss` (issuer) o `aud` (audience) no se valida, un token emitido para una aplicación puede colarse en otra distinta del mismo proveedor.

Defensa: validar siempre `iss` y `aud` contra el valor esperado de la aplicación concreta.

6. Tokens sin expiración o expiración eterna

Sin `exp` o con `exp` muy lejano (años), un token robado vale para siempre. Combinado con almacenamiento inseguro en localStorage, un XSS exfiltra el token y el atacante mantiene acceso indefinido.

Defensa: `exp` corto (15-60 min para

access tokens), refresh tokens con vida más larga gestionados aparte, revocación gestionada por blacklist o por rotación de claves.

7. Firma HS256 con la palabra "secret" como clave

Sigue apareciendo en código en producción más a menudo de lo razonable. Defensa: revisión sistemática de configuración antes de pasar a producción + escáner SAST con regla específica.

JWT vs sesión tradicional con cookie

Las dos aproximaciones tienen tradeoffs reales:

Característica	JWT
Almacenamiento servidor	Sin est

Escalabilidad horizontal	Trivial (
Revocación	Difícil (t
Tamaño	0,5-2 K
Riesgo en cliente	Token c
Renovación	Refresh

Decisión razonable:

- **JWT**: APIs públicas, microservicios, SSO, mobile apps. Vale la pena el esfuerzo defensivo.
- **Sesión tradicional**: aplicaciones web monolíticas con un servidor o pocas instancias detrás de un load balancer. Más simple, menos vectores de ataque.

Buenas prácticas de seguridad con JWT

Las que cualquier auditoría espera ver implementadas:

1. **Lista blanca de algoritmos**. Validación estricta del `alg`. Rechazar `none`. No mezclar simétrico y asimétrico.
2. **Secret fuerte**. 32 bytes aleatorios mínimo para HS256, gestionado en KMS o secret manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
3. **Rotación de claves**. `kid` versiona la clave; el servidor mantiene un set de claves activas + historial reciente. Rotar al menos cada 90 días.
4. **Validar** `iss`, `aud`, `exp`, `iat`, `nbf`. Cada uno frena un vector. No es opcional.
5. `exp` **corto**. 15-60 min para access tokens. Refresh token aparte.
6. **Almacenamiento cliente**. HttpOnly cookies con SameSite=Strict mejor que localStorage (que es accesible a XSS). Si se usa localStorage, asumir que un XSS roba el token.
7. **Sin secretos ni datos sensibles**

en el payload. Es legible por cualquiera con el token.

8. **Revocación operativa**. Lista negra de tokens, rotación inmediata de la clave en caso de compromiso, logout que invalida refresh tokens.
9. **Logs sin tokens**. No loggear el JWT completo. Si hace falta debugging, sólo el `sub` y el `jti`.
10. **Auditoría externa periódica**. Pentesting que cubra explícitamente JWT con OWASP API Security Top 10 categoría 2 (Broken Authentication).

JWT y compliance

- **PCI DSS v4.0 (req. 8 y 6.4.6)**. La gestión de tokens forma parte de la autenticación robusta exigida sobre el CDE. Cualquier cambio significativo requiere prueba técnica.
- **RGPD (artículo 32)**. Medidas técnicas apropiadas. Tokens sin

expiración ni revocación dificultan el ejercicio del derecho de supresión.

- **NIS2 (artículo 21)**. Eficacia de medidas técnicas en autenticación.
- **eIDAS 2** (futuro EU Digital Identity Wallet). Uso intensivo de claims firmados; los formatos JWT y Verifiable Credentials se cruzan.

Preguntas frecuentes

¿JWT es seguro por sí mismo?

Sí, si se implementa correctamente. El estándar es sólido (RFC 7519). Los problemas vienen casi siempre de implementación: `alg: none` permitido, secret débil, no validar `aud` / `iss` / `exp`, o almacenamiento inseguro en cliente. JWT mal implementado es una de las categorías de bugs más frecuentes

en pentest API.

¿Cuál es la diferencia entre JWT y OAuth 2.0?

OAuth 2.0 es un protocolo de autorización (cómo se obtiene un token); JWT es un formato de token (qué pinta tiene). OAuth 2.0 puede usar JWT como formato o usar tokens opacos. OpenID Connect (capa de identidad sobre OAuth 2.0) sí usa JWT obligatoriamente para los `id_token`.

¿Conviene cifrar el payload (JWE) en lugar de sólo firmarlo (JWS)?

Por defecto basta JWS (firma). Sólo conviene JWE (cifrado) si el payload contiene información sensible que no debe llegar al cliente, lo cual ya es señal de que probablemente no debería estar ahí. La regla práctica: que el payload contenga sólo identificadores y permisos, datos sensibles fuera del token. Así la

firma es suficiente.

¿Cuánto debe durar un JWT?

Para access tokens: 15-60 minutos como punto de partida. Tokens más largos amplifican el daño de un robo. Para refresh tokens: días o semanas, almacenados en HttpOnly cookies o en backend con rotación al usar.

¿Cómo se revoca un JWT antes de su expiración?

Tres aproximaciones: (1) blacklist en cache (Redis, Memcached) con los `jti` revocados hasta su `exp`; (2) versionado por usuario (incrementar un contador en BBDD que se compara con un claim del token); (3) rotación de la clave de firma cuando hay compromiso masivo. La revocación granular es el principal motivo por el que algunos equipos prefieren sesión tradicional sobre JWT.

¿Dónde guardo el JWT en el cliente: cookie o localStorage?

Cookie HttpOnly + Secure + SameSite=Strict para aplicaciones web tradicionales. localStorage sólo cuando lo exija la arquitectura (algunos SDKs móviles), aceptando que un XSS exfiltra el token. Nunca en sessionStorage para tokens persistentes.

Recursos relacionados

- Pentesting de APIs REST y GraphQL: metodología técnica para auditar APIs que usan JWT.
- Pentesting de aplicaciones web: cómo se auditan los flujos de autenticación con JWT en web.
- Pentesting móvil iOS y Android: JWT en almacenamiento Keychain/Keystore y bypass de validación.

- OWASP Top 10 explicado para empresas: la categoría A07 Identification and Authentication Failures cubre la mayoría de bugs JWT.
- Auditoría web: guía completa: qué incluye una auditoría web profesional, donde JWT es uno de los focos clave.

JWT en Secra

En Secra auditamos implementaciones JWT como parte habitual de nuestros pentesting web, móvil y API. Cubrimos análisis de algoritmos, gestión de secretos, validación de claims, almacenamiento en cliente, rotación y revocación. Si tu organización quiere validar la postura de JWT en una aplicación crítica antes de un release o una certificación PCI/NIS2, escríbenos a través de contacto o consulta el servicio de auditoría web y móvil.

SOBRE EL AUTOR

Equipo de Secra Solutions

Ethical hackers certificados OSCP, OSEP, OSWE, CRTO, CRTL y CARTE, con más de 7 años de experiencia en ciberseguridad ofensiva. Autores de los CVE-2025-40652 y CVE-2023-3512.

Conoce al equipo →

Nuestra investigación →

Compartir artículo

Artículos Relacionados

defensiva

SOC as a Service: guí

Qué es SOC as a Service, modelos de externalización

defensiva

Tipos de phishing:

Los 12 tipos de phishing más comunes (spear,

defensiva

Qué es SHA-256: funció

Qué es SHA-256, cómo funciona como función has

2026-05-13 15 min

2026-05-12 16 min

2026-05-11 12 min

Proporcionamos soluciones de ciberseguridad líderes en la industria para proteger su negocio contra amenazas en evolución.

Secra Solutions, S.L.

Madrid, España

contacto@secra.es

+34 711 200 544

Empresa

- Inicio
- Quiénes Somos
- Metodología
- Investigación
- Blog
- Contacto

Soluciones

- Ciberseguridad Ofensiva
- Consultoría GRC
- Ciberseguridad Gestionada
- Formación
- SSDLC & DevSecOps
- Soluciones PYMEs

Empresa Certificada

CERTIFICACIÓN DE CONFORMIDAD CON EL ens Esquema Nacional de Seguridad Categoría MEDIA RD 311/2022

incibe emprende CIBEREMPRENDE

Aceptar todas Configurar Rechazar

Utilizamos cookies propias y de terceros para mejorar tu experiencia de navegación y analizar el tráfico. Más información